

ResNET 논문 리뷰

Abstract

- 문제의식: 네트워크가 깊어질수록 학습(최적화)이 어려워진다.
- 제안: 레이어들이 원하는 함수 $H(x)$ 를 직접 맞추게 하지 않고, **입력 대비 변화량인 $F(x)=H(x)-x$ 를 학습하게 만든다.**
- 결과적으로 블록 출력은 $H(x)=F(x)+x$ 꼴이 되고, 이 구조가 매우 깊은 모델에서도 학습을 안정적으로 만든다는 주장이다.
- 실험 요약: ImageNet에서 152-layer까지 학습해 성능을 보였고, COCO detection에서도 개선이 있었다.

1. Introduction

- 배경: 딥 CNN은 깊이가 늘면서 성능이 좋아지는 흐름이 있었고, 그래서 더 깊게 만드는 것이 자연스러운 방향이었다.
- 기존의 문제: 과거에는 vanishing/exploding gradient로 아예 학습이 시작이 안 된다는 것이 큰 문제였는데, 초기화/BN 같은 기법으로 수십 층은 학습이 가능해졌다.
- 그러나 여기서 논문이 제기하는 또 다른 문제: **degradation problem**
 - 깊이를 더 늘리면 test 성능이 포화되는 정도가 아니라 오히려 나빠지기도 한다.
 - 이게 overfitting이라면 training error는 더 낮아져야 하는데, 실제로는 더 깊은 plain net이 training error 자체가 커진다.
 - 즉 데이터에 과하게 맞춘 문제가 아니라 **최적화가 잘 안 되는 문제**라는 결론으로 이어진다.
- 이것이 문제인 이유:
 - 깊은 모델은 얇은 모델을 포함할 수 있어야 한다.
 - 추가된 레이어들이 그냥 identity처럼 동작하면, 깊은 모델이 얇은 모델 성능보다 최소한 나빠질 이유가 없다.

- 그런데 실제로는 그 쉬운 해조차 잘 찾지 못해서 성능이 떨어진다는 것은 구조적으로 **optimization이 어려운 형태**라는 뜻이다.
- 그래서 제안하는 관점의 전환:
 - identity를 직접 구현하려고 하지 말고, identity에서 얼마나 달라지는지, 즉 **residual을 학습**하자는 것이다.
 - $H(x)$ 대신 $F(x)=H(x)-x$, 출력은 $F(x)+x$
 - 이걸 **shortcut(skip) connection**으로 구현한다고 예고한다.

2. Related Work

- (1) Residual representations
 - 이미지 표현에서도 원본 자체보다 **기준 대비 차이(residual)**를 쓰는 방식이 강력했다.
 - 예: VLAD / Fisher Vector 계열은 residual 기반 인코딩이 성능에 유리했던 대표 사례로 언급된다.
- (2) 최적화/수치해석의 관점
 - multigrid, preconditioning처럼 문제를 잘 재정식화(reformulation)하면 수렴이 빨라진다.
 - ResNet도 본질적으로 **학습할 대상을 residual로 바꾸는 재정식화**라는 점에서 같은 철학을 공유한다고 할 수 있다.
- (3) skip/shortcut connection 계열
 - shortcut/skip 자체는 예전에도 여러 형태로 등장했고(우회 경로, 보조 분기 등), 학습 안정화에 쓰인 사례가 있다.
 - 특히 Highway Networks는 skip에 gate를 두지만, ResNet은 parameter-free identity shortcut을 강조한다.
 - 즉 ResNet은 항상 identity 경로가 열려 있고, 레이어는 그 위에 residual만 얹는 구조라는 점이 차별점이다.

3. Deep Residual Learning

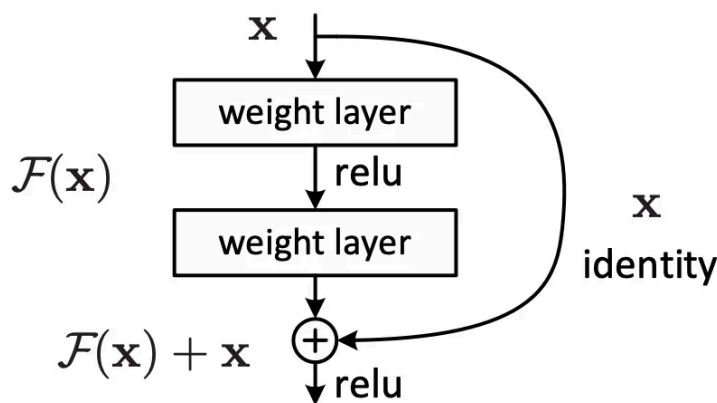
3.1 Residual Learning

- 우리가 원하는 최적의 함수는 $H(x)$
- 신경망이 복잡한 함수를 근사할 수 있다면, 그 함수의 잔차 $H(x) - x$ 도 근사할 수 있을 것이다.
- 재정의 : $H(x)$ 를 직접 학습하는 대신, $F(x) := H(x) - x$ 를 학습하자. 따라서 원래 학습하려던 함수 $H(x)$ 는 $F(x) + x$ 가 된다. 항등매핑이 최적의 정답에 가깝다면? 훨씬 계산이 쉬워진다.

3.2 Identity Mapping by Shortcuts

$$y = F(x, \{W_i\}) + x$$

- F 는 학습해야 할 잔차 매핑
- $F = W_2\sigma(W_1x)$ 처럼 보통 2개의 층을 통과 → 마지막에 한번더 ReLU 적용



- $F(x)$ 와 x 를 더하려면 두 벡터의 차원(채널 수 등)이 같아야 함. 두가지 방법
 1. Identity Mapping : zero-padding 활용. 파라미터가 들지 않음
 2. Projection Shortcut : 1×1 Convolution 같은 연산(W_s)을 통해 차원을 맞춰줌
 $y = F(x, \{W_i\}) + W_s x$ 와 같은 식이 됨.

3.3 Network Architectures

VGGNet의 철학을 계승하여 ImageNet용 모델을 설계

- **Plain Network (비교군) 설계 방식** : VGGNet처럼 주로 3×3 필터를 사용
 - 출력 피쳐 맵 크기가 같으면, 필터 개수도 똑같이 유지
 - 피쳐 맵 크기가 절반으로 줄어들면, 연산량 보존을 위해 필터 개수를 2배로 늘린다. 이때 크기를 줄이는 downsampling 은 풀링 대신 stride 2 convolution을 사용.
 - 마지막에 global average pooling 을 써서 연산량 줄임
- **Residual Network** : ResNet-34는 층이 훨씬 깊음에도 불구하고, VGG-19보다 훨씬 적은 연산량
 - 실선 : 차원 같은 경우
 - 점선 : 차원 늘어날때는 zero padding이나 projection (1×1 Conv) 주로 사용

3.4 Implementation

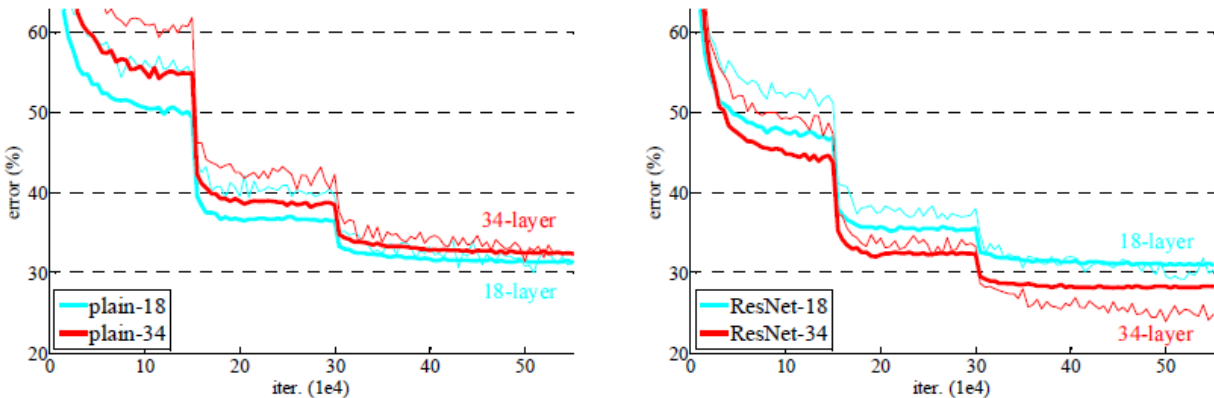
- 이미지 전처리 : 이미지의 짧은 변을 256~480 사이로 랜덤하게 리사이징(Scale augmentation)하고, 그 안에서 224×224 크기로 자르기
- Batch Normalization : 모든 Conv 바로 다음, 활성화 함수(ReLU) 바로 전에 **BN**을 적용 (딥러닝 학습 안정화) → 규제 효과로 dropout은 따로 사용하지않음.
- 가중치 초기화(Weight Initialization) 사용, SGD (Mini-batch 256) 사용, Learning Rate는 0.1에서 시작, 에러가 줄지 않으면 1/10로 줄임. Weight Decay: 0.0001 (가중치 규제), Momentum: 0.9 (이전방향 90% 반영)

4. Experiments

4.1 ImageNet Classification

ImageNet 2012 데이터셋(1000개의 class가 있는 데이터셋임)을 사용하여 Plain Network 와 ResNet의 성능을 비교 분석함.

1. Plain Network vs. ResNet (실제로 Degradation가 존재하는지)

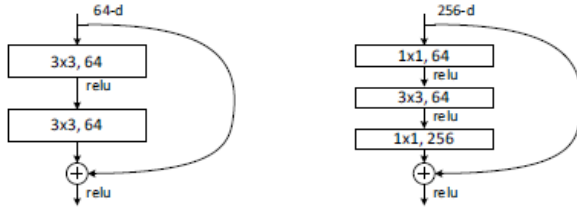


- Plain Network: 34-layer 모델이 18-layer 모델보다 Training Error가 더 높음.
 - Vanishing Gradient 문제는 batch normalization으로 해결함. 따라서 이는 최적화 Optimization의 실패임.
- ResNet: 34-layer 모델이 18-layer 모델보다 에러가 더 낮음.
 - 깊이가 깊어질수록 성능이 향상됨을 확인.
 - 초기 수렴 속도도 ResNet이 더 빠름.

2. Identity vs. Projection Shortcuts

- Option A (Zero-padding): 파라미터 증가 없음.
- Option B (Projection): 차원이 늘어날 때만 1 X 1 Convolution 사용.
- Option C (All Projection): 모든 shortcut에 1 X 1 Convolution 사용.
- 결과: $C > B > A$ 순으로 성능이 좋지만, 차이는 미미함.
 - C는 파라미터가 너무 많아지므로, 메모리 효율을 위해 Option B를 주로 사용함.

3. Deeper Bottleneck Architectures



- 학습 시간과 연산 효율을 위해 50-layer 이상의 모델에서는 Bottleneck Design 도입.
- 구조: 1 X 1 (차원 축소) → 3 X 3 (연산) → 1 X 1 (차원 복원)
 - 기존 2-layer 블록을 3-layer 블록으로 변경.
 - 1 X 1 Conv가 Bottleneck을 만들어 3 X 3 Conv의 입출력 채널 수를 줄여줌.
 - 이 구조에서는 parameter free, Identity Shortcut이 중요함 (Projection을 너무 활용하면 모델이 너무 커짐).

4. Result 요약

- ResNet-152: VGG-19보다 깊이는 8배(152층)지만 복잡도(FLOPs)는 오히려 낮음.
- 단일 모델 Top-5 Error: 4.49%
- 앙상블(Ensemble) 모델 Top-5 Error: 3.57% (ILSVRC 2015 우승)

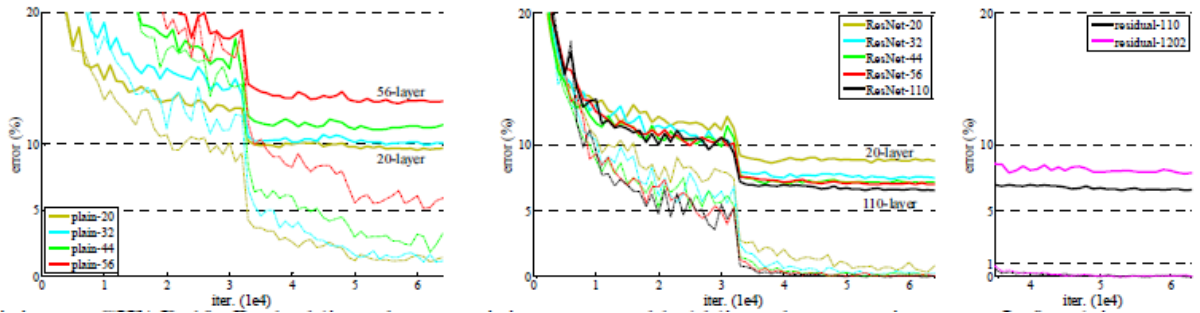
4.2 CIFAR-10 and Analysis

CIFAR-10 데이터셋을 통해 극도로 깊은 네트워크(1000층 이상)의 동작을 분석.

1. Deep ResNet 분석

- n=3, 5, 7, 9 일 때 각각 20, 32, 44, 56-layer, 레이어가 깊어질수록 정확도 상승.
- 110-layer ResNet: 1.7M 파라미터만으로 SOTA 달성.
- 1202-layer ResNet: 1000층이 넘어도 Optimization에는 문제가 없으나, 데이터셋 크기에 비해 모델이 너무 커서 약간의 Overfitting 발생 (Test error가 110-layer보다 소폭 높

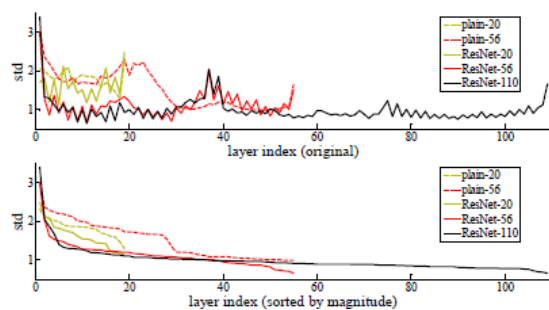
음).



- CIFAR-10에서의 결과. Plain Net은 깊을수록 training error가 높지만, ResNet은 깊을수록 낮아짐을 확인 가능.

2. Layer Response Analysis

- 가설: 잔차 함수 $F(x)$ 는 0에 가까울 것이다.
- 결과: ResNet의 레이어 출력값들의 표준편차를 분석결과, Plain Net보다 훨씬 0에 가까운 값을 가짐.
- 깊이가 깊어질수록(20 → 56 → 110), 개별 레이어는 신호를 덜 수정하는 경향이 있음 (Identity function에 더 가까워짐)



- 레이어 출력값의 표준편차 그래프. ResNet이 Plain Net보다 값이 작음. 이는 잔차 학습이 의도한 대로 작동하고 있음을 통계적으로 증명.

4.3 Object Detection on PASCAL and MS COCO

Classification뿐만 아니라 Detection에서도 Generalization 성능이 좋은지 검증.

- Faster R-CNN 프레임워크 사용.
- 기본 백본인 VGG-16을 ResNet-101로 교체.
- 결과: COCO 데이터셋에서 mAP가 6.0% 향상 (상대적으로 28% 성능 향상).
- 이는 오직 모델의 Feature 표현력(Representation)이 좋아졌기 때문임.